

```

%% #####
% (c) Thomas Abmayr, HM, FK08, WS202X
% -----
% Content: Visual Odometrie and 3D Reconstruction / Modul Mobile Mapping
% -----
% Data: Tsukuba University's CVLAB
% [1] "Realistic CG Stereo Image Dataset with Ground Truth Disparity Maps",
%      Martull et al., TrackMark 2012.
% [2] "Towards a Simulation Driven Stereo Vision System", Peris et al.,
%      ICPR2012.
% #####

clear all;
close all;

%% #####
% Schritt 0: init
% #####

%% const ==> bounds
SIZE_SLIDING_WND = 10;
BASELINE = 0.1; % 10cm

%% P6 + P8 ==> load images
images = imageDatastore('C:\HM\Data\Tsukuba\NTSD-200\mod\left');
images_r = imageDatastore('C:\HM\Data\Tsukuba\NTSD-200\mod\right');
images_d = imageDatastore('C:\HM\Data\Tsukuba\NTSD-200\mod\disp');

%% p6: Create an empty viewSet object to manage the data assoc. with each view.
vSet_est = viewSet;
vSet_real = viewSet;

%% P6 first view active and gray image

```

```

Irgb_1 = readimage(images, 1);
Igrey_1 = rgb2gray(Irgb_1);

%% P6: init cam params
K = [615 0 320; 0 615 240; 0 0 1]';
cameraParams = cameraParameters('IntrinsicMatrix', K);

%% P6: undistort
Igrey_1 = undistortImage(Igrey_1, cameraParams);

%% P6: apply SURF
viewId = 1;
points = detectSURFFeatures(Igrey_1);
% points = points.selectStrongest(200);
[feat1, valid_corners_1] = extractFeatures(Igrey_1, points);

%% P6: Add the first view (estimated) // P8 / P10: Add the first view (real).
vSet_est = addView(vSet_est, viewId, 'Points', valid_corners_1, 'Orientation', eye(3), 'Location', [0 0 0]);
vSet_real = addView(vSet_real, viewId, 'Points', valid_corners_1, 'Orientation', eye(3), 'Location', [0 0 0]);

%% P6
figure(1);
subplot(3,3,1);
imshow(Igrey_1);
hold on;
plot(points, 'showOrientation', true);
title('First Image');

%% P8: load vSet_real (transf. from GT)
% load('vSet_real.mat');

%% P11: load ground truth data ==> +[0,0,5] transl. to left image
load('C:\HM\praesentationen\MSNav\Praktikum\WS20XX_MobMap\P8_Wiederholung_2v5\source\GroundTruth_mod.mat');
groundTruthPoses = groundTruthPoses_ii;

```

```

%% P9: Visualise Traj.
figure(10);
title('Trajectory (real, estimated w/o ba, estimated w ba)');
axis([-50, 10, -50, 30, -50, 150]);
camroll(180);
view(gca, 3);
set(gca, 'XDir', 'reverse');
set(gca, 'CameraUpVector', [0, -1, 0]);
grid on
xlabel('X (cm)');
ylabel('Y (cm)');
zlabel('Z (cm)');
hold on;
plotCamera('Size', 3, 'Location', vSet_est.poses.Location{1}, 'Orientation', vSet_est.poses.Orientation{1}, ...
    'Color', [0.7, 0.7, 0.7], 'Opacity', 0);

%% #####
% Schritt 1: canc Traj
% #####
for viewId = 2 : SIZE_SLIDING_WND

    %% P6: load scd image
    Irgb_2 = readimage(images, viewId);
    Igrey_2 = rgb2gray(Irgb_2);

    Irgb_2_sp = readimage(images_r, viewId);
    Igrey_2_sp = rgb2gray(Irgb_2_sp);

    figure(2);
    imshow( [Irgb_2, Irgb_2_sp] );

    %% P6: undistort
    Igrey_2 = undistortImage( Igrey_2, cameraParams);

```

```

Igrey_2_sp = undistortImage( Igrey_2_sp, cameraParams);

%% P6: Feature Extraction im zweiten Bild
figure(1);
subplot(3,3,2);
title('Ith Image');
corner2 = detectSURFFeatures(Igrey_2);
[feat2, valid_corners_2] = extractFeatures(Igrey_2, corner2);

imshow(Igrey_2);
hold on;
plot(valid_corners_2, 'showOrientation', true);

%% P6: Feature Matching zwischen beiden Bildern
figure(1);
subplot(3,3,3);
indexPairs = matchFeatures(feat1, feat2) ;
matchedPoints1 = valid_corners_1(indexPairs(:,1), :);
matchedPoints2 = valid_corners_2(indexPairs(:,2), :);
showMatchedFeatures(Igrey_1, Igrey_2, matchedPoints1, matchedPoints2);
legend('matched points 1', 'matched points 2');
title('Matched Points');

%% P6: Add the current view to the view set
vSet_est = addView(vSet_est, viewId, 'Points', valid_corners_2);

%% P6: Store the point matches and pose between the previous and curr. view.
vSet_est = addConnection(vSet_est, 1, viewId, 'Matches', indexPairs);

%% P9: F Matrix berechnen
[F, inliersIndex_f, status_f] = estimateFundamentalMatrix(...
    matchedPoints1, matchedPoints2, 'Method', 'RANSAC', 'NumTrials', 2000);
Rank_f = rank(F);
ev0 = eig(F);

```

```

%% P9: Outlier suchen und inlier plotten
figure(1);
subplot(3,3,4);
inlierPoints1 = matchedPoints1(inliersIndex_f);
inlierPoints2 = matchedPoints2(inliersIndex_f);
showMatchedFeatures(Irgb_1, Irgb_2, inlierPoints1, inlierPoints2);
legend('inlierPoints 1', 'inlierPoints 2');
title('Inlier Points');

%% P9: test inliers
N=1;
for i=1 : inlierPoints2.Count()
    val = [inlierPoints2.Location(i,:),1]*F*[inlierPoints1.Location(i,:),1]';
    if (abs(val)>0.8)
        test_out(N) = val;
        N=N+1;
    end
end

%% P9: E Matrix extrahieren
K = cameraParams.IntrinsicMatrix';
%E = inv(inv(K)')*F*inv(K);
E = inv(inv(K)')*F*K;
F_Test = inv(K)'*E*inv(K);
F_diff = (F_Test - F);
F_norm = norm(F_Test - F);
Rank_E = rank(E);
[U,D,V] = svd(E);
ev = eig(E);

%% P9: E Matrix mit inliern berechnen
[E2, inliersIndex_e, status_e] = estimateEssentialMatrix(matchedPoints1, ...
    matchedPoints2, cameraParams);

```

```

F_Test2 = inv(K)'*E2*inv(K);
F_diff2 = (F_Test2 - F);
F_norm2 = norm(F_Test2 - F);
Rank_E2 = rank(E2);
[U2,D2,V2] = svd(E2);
ev2 = eig(E2);

%% P9: update vset
vSet_est = updateConnection(vSet_est, 1, viewId, 'Matches', indexPairs(inliersIndex_e,:));

%% P10: pose berechnen
[ori12, loc12] = relativeCameraPose(E2, cameraParams, ...
    matchedPoints1(inliersIndex_e,:), matchedPoints2(inliersIndex_e,:));

%% P10: calc scale from groundTruth (=GPS, odometrie o.ä.)
tmp2 = groundTruthPoses.Location{viewId};
tmp1 = groundTruthPoses.Location{1};
fac = norm(tmp1-tmp2, 2);
loc12 = fac*loc12;

%% P10: Add the current view to the view set
vSet_est = updateView(vSet_est, viewId, 'Orientation', ori12, 'Location', loc12);

%% P10: plot traj and camera
figure(10);
tmp = vSet_est.poses.Location{viewId};
scatter3(tmp(:,1), tmp(:,2), tmp(:,3), 'g');
hold on;
plotCamera('Size', 3, 'Location', vSet_est.poses.Location{viewId}, 'Orientation', vSet_est.poses.Orientation{viewId}, 'Color', [0.7, 0.7, 0.7], 'Opacity', 0);

%% P11: calc tranformation (real): left_i ==> center && center ==> left_1
Mw2ref(1:4,1:4) = eye(4);
Mw2ref(1,4) = 5;

```

```

    Ti2w = groundTruthPoses.Location{viewId};
    Ri2w = groundTruthPoses.Orientation{viewId};
    Mi2w(1,1:4) = [Ri2w(1,1), Ri2w(1,2), Ri2w(1,3), Ti2w(1)];
    Mi2w(2,1:4) = [Ri2w(2,1), Ri2w(2,2), Ri2w(2,3), Ti2w(2)];
    Mi2w(3,1:4) = [Ri2w(3,1), Ri2w(3,2), Ri2w(3,3), Ti2w(3)];
    Mi2w(4,1:4) = [0, 0, 0, 1];
    Mi2ref = Mw2ref * Mi2w;

    %% P11: add to viewset (real)
    vSet_real = addView(vSet_real, viewId, 'Orientation', Mi2ref(1:3,1:3)', 'Location', Mi2ref(1:3,4)); % 'normal' ==>
Matlab Notatio

    %% P10: plot camera (real)
    tmp = vSet_real.poses.Location{viewId};
    scatter3(tmp(:,1), tmp(:,2), tmp(:,3), 'g');
    hold on;
    R = vSet_real.poses.Orientation{viewId};
    [U,D,V] = svd(R);
    D1 = eye(3);
    D1(3,3) = det(U*V);
    R = U*D1*V';
    % plotCamera('Size', 3, 'Location', vSet_real.poses.Location{viewId}, 'Orientation', R, 'Color', [0.7, 0.7, 0.7],
'Opacity', 0);

    %% ende
    pause(0.3);

end

save('vSet_est.mat', 'vSet_est' );
%% #####
% Schritt 2: Bundle Adjustment ==> Wo 11
% #####

```

```

% P11: bundle adjustment for sliding window (==>multiple view geometrie)

% Find point tracks spanning multiple views.
tracks = findTracks(vSet_est);
% Get camera poses for all views
camPoses = poses(vSet_est);
% calculate initial locations for the 3-D world points.
[xyzPoints, reprojErr] = triangulateMultiview(tracks, camPoses, cameraParams);
% Refine camera poses using bundle adjustment.
[xyzPoints_refined, camPoses] = bundleAdjustment(xyzPoints, tracks, camPoses, cameraParams, 'PointsUndistorted', true,
'FixedViewIDs', 1, 'MaxIterations', 300);
% Update view set.
vSet_est = updateView(vSet_est, camPoses);

% plot camera nach ba
figure(10);
for viewId=1 : SIZE_SLIDING_WND
    tmp(viewId,1:3) = vSet_est.poses.Location{viewId};
    plotCamera('Size', 3, 'Location', vSet_est.poses.Location{viewId}, 'Orientation', vSet_est.poses.Orientation{viewId},
'Color', 'b', 'Opacity', 0);
end
scatter3(tmp(:,1), tmp(:,2), tmp(:,3), 'b');
hold on;

figure(11);
ptCloud = [ ];
ptCloud = pointCloud(xyzPoints_refined);
pcshow(ptCloud, 'MarkerSize', 50, 'VerticalAxis', 'Y', 'VerticalAxisDir', 'Down');

% save('D:\HM\praesentationen\MSNav\Praktikum\WS20XX_MobMap\P8_Wiederholung_2v5\source\vSet_real.mat', 'vSet_real');
%% #####
% Schritt 3 ==> Wo 8
% #####

```

```
% mobMap_denseMatching(images, images_d, vSet_est, SIZE_SLIDING_WND, K, BASELINE);  
mobMap_denseMatching(images, images_d, vSet_est, SIZE_SLIDING_WND, K, BASELINE );
```